Completing the Successor-Seeded Scalar Basis for iPRA

A predecessor construction resolving an open candidate in the supplied drafts

Research prepared for Vladimir Reshetnikov

November 10, 2026

Abstract

The shared-continuation draft proposes a numerical language consisting of zero and a scalar iterator that starts at the successor of its initializer. It explicitly leaves open the construction of a predecessor in this language. We complete that candidate. A shifted factor map yields an internally correct comparator; a saturated iteration yields a capped predecessor; a fixed three-input correction yields a genuine predecessor. Every arithmetic induction in the new bootstrap has a single literal equation as its invariant. No predecessor, successor cancellation, ordinary initialization, pairing, or general selector is used to manufacture the predecessor.

The resulting basis has two numerical constructors, two computation schemas, and one changing update variable. With the explicit native logical rules given below it is bi-interpretable and equiconsistent with full first-order intuitionistic primitive recursive arithmetic. Both interpretations use the entire numerical domain and actual equality, and both numerical composite isomorphisms are the identity.

The mathematical completion and the implementation comparison are separate claims. The supplied complete checker improves the inspected byte frontier, but does not establish a simultaneous improvement in every source-size coordinate, absolute minimality, or global priority. The equivalence proof is a written proof with independent assistant review and executable diagnostics; it is not a Lean or Rocq formalization.

Contents

1 What is new, and what has actually been minimized **3**
1.1 The resolved obligation . . . 3
1.2 Complete-checker comparison . . . 3

2 The entire primitive theory **4**
2.1 Numerical language . . . 4
2.2 Native formulas and every logical rule . . . 4
2.3 The equivalent computation-supply presentation . . . 5

3 Native bootstrap: cases before a full selector **5**
3.1 Resources available without a numerical selector . . . 6
3.2 Functional finite ranges do not assume branch equations . . . 6
3.3 Audit of the bootstrap and completion of the bridge . . . 8

4 A successor-seeded scalar basis with two computation schemas **9**
4.1 Syntax, computations, and the precise resources used . . . 9
4.2 A partial selector and shifted addition . . . 10
4.3 Two finite cycles . . . 10
4.4 The shifted ramp and factor map . . . 11
4.5 A comparator without stripping an initializer . . . 12
4.6 Saturation and a finite correction give true predecessor . . . 13
4.7 Consequences and the bi-interpretation boundary . . . 14

5 Recovering full first-order intuitionistic logic **15**
5.1 Gates, padding, and formula selection . . . 15
5.2 Explicit logical macros and their rules . . . 16

6 The two interpretations and their composites **16**

7 Complete executable checkers and their correspondence **18**
7.1 The complete byte profile . . . 18
7.2 The complete AST profile . . . 19
7.3 Public protocol and exact sorted decoding . . . 19
7.4 The AST profile's additional local aliases . . . 20
7.5 Every instruction branch . . . 21
7.6 Why generated J equations are sound . . . 22
7.7 Executable evidence and its limits . . . 23

8 Further research questions **24**

A From literal-equation induction to full intuitionistic PRA **24**
A.1 The exact hypotheses . . . 24
A.2 Constructor separation and two elementary tests . . . 25
A.3 Four elementary uses of equation induction . . . 26
A.4 An equation invariant that reflects actual equality . . . 27
A.5 Compilation of all quantifier-free formulas . . . 28
A.6 Recovery of induction and the full theory . . . 29

B Pairing, primitive recursion, and internal numerical composites **29**
B.1 Pairing from a bounded square-root search . . . 30
B.2 Compiling each primitive recursion . . . 30
B.3 Why both numerical round trips are internal . . . 31

1 What is new, and what has actually been minimized

1.1 The resolved obligation

The comparison is pinned to ProveIt commit

`2fdcf5bf988bead821c416649f713f98b7ed8776`.

The current directory contains nine direct ZIP archives. In `ipra_shared_continuation_2026-11-10.zip`, the report's section "A genuinely smaller numerical grammar: the precise unresolved step" states exactly the successor-seeded scalar candidate studied here and proves a conditional initialized-iterator compiler. Its unresolved obligation is a finite term P and an internal proof of $P(Sa) = a$ [1]. The present work supplies that term and proof, including $P(0) = 0$. It therefore completes an existing proposed basis rather than claiming to have invented its syntax.

The new steps are the shifted affine alignment of the factor-map construction, its whole-domain comparator proof, and the finite correction of a capped predecessor. The earlier ordinary scalar bootstrap already uses finite cycles, a factor map, and saturation; that strategy is credited to the supplied development[2]. Related restricted-iteration and seed-shifting methods occur in Severin[8]. This limited literature check establishes context, not a global priority claim.

Numerical basis	Constructors	Computation schemas	Update variables
Initialized scalar $0, S, I$	3	2	1
Counter-aware successor $0, R$	2	2	2
Completed scalar $0, J$	2	2	1

The counts omit variables and count a binder-forming operation as one constructor. The R comparison is the existing successor-recursion proposal[3]. Two numerical constructors alone are not a proof-checker lower bound: argument structure, logic, validation, and encoding also cost code.

1.2 Complete-checker comparison

Complete executable	Tokens	AST nodes	Normalized bytes
Prior generated-assumption checker	281	318	685
Prior AST-oriented checker	283	315	710
New J: direct adaptation	283	320	687
New J: byte profile	284	322	**681**
New J: AST profile	282	**313**	707
New J: total-input AST profile	287	**314**	711

The new separate-profile envelope is 282/313/681, compared with the prior 281/315/685. It improves AST size by two nodes and normalized length by four bytes. It does not improve the token minimum. The AST profile's additional raw aliases and its reliance on CPython's finite recursion limit for malformed nonempty strings are documented in Section 7. The additional total-input profile improves AST size by one node while retaining termination on every finite malformed input in the ideal unbounded-resource model. Its separate-profile envelope with the direct and byte versions is 283/314/681. The audit recursively includes embedded source archives and complete programs stored in reference directories. It found 415 Python source occurrences, 164 distinct source hashes, and no

parse errors. Fourteen distinct smaller-coordinate utilities were inspected and excluded because they are builders, measurers, integrity verifiers, or test runners calling another checker. The complete measured frontier is consequently 281 normalized tokens, 315 AST nodes, and 685 normalized bytes; different existing programs attain these coordinates.

The metric removes comments and actual docstrings, applies `ast.unparse`, and measures the entire module, including JSON input, scope checks, inference checks, rejection, and verdict output. Python and its standard JSON parser are common runtime infrastructure. The supplied normalizer and frozen controls reproduce the comparison under CPython 3.12.14. An observed minimum among implementations is an attained upper bound on the possible checker cost, not a proved mathematical lower bound.

The byte saving is small and partly a Python source representation change. The substantive mathematical result is the resolved predecessor obligation. We do not attribute a representation-only saving to the arithmetic theorem, nor claim that a single supplied program beats all three current coordinate minima.

2 The entire primitive theory

Here iPRA is the full first-order intuitionistic theory of fixed finite primitive-recursive descriptions, their equations, the successor boundary laws, and quantifier-free induction. The atomic-induction presentation is standard[7]; the explicit equality-reflection argument in Appendix A verifies the relevant equivalence here. It is important to retain the full first-order logical interface. Equivalence only with the quantifier-free equational fragment would not satisfy the requested bi-interpretability condition.

2.1 Numerical language

The finite staged terms are

$$t ::= x \mid 0 \mid J(n, a; u.h), \qquad Sa := J(0, a; u.0), \qquad 1 := S0.$$

Only u in the update h is bound. Its remaining parameters are fixed throughout that iteration. The numerical axioms, available in any retained context, are exactly

$$J(0, a; u.h) = Sa, \qquad J(Sn, a; u.h) = h[J(n, a; u.h)/u]. \tag{J0, JS}$$

Every substitution is capture-free. Equivalently introduce one ordinary function symbol $j_h(n, a, \vec{p})$ for each fixed earlier-stage description $h(u, \vec{p})$. In particular the zero-body description defines successor. There is no object-level quantification over descriptions, no higher-type function sort, and no total universal evaluator for all PR descriptions.

2.2 Native formulas and every logical rule

Formulas and reusable contexts have grammar

$$A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A, \qquad \Gamma \text{ a finite reusable list of such formulas.}$$

The variables x, y are distinct. Both are bound by D; only y is bound by Q. Their intended ordinary meanings are

$$(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ(y), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

Besides the two computation schemas, the complete primitive inference inventory is

$$\frac{}{\Gamma \vdash A}\ (A \in \Gamma), \qquad \frac{\Gamma \vdash s = t \quad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}, \qquad \frac{\Gamma \vdash \mathsf{Q}_y B \quad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}. \tag{Hyp, Tr, QE}$$

The transport motive F is any fixed native formula template. The paired elimination and its two introduction choices are

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x,y) \quad \Gamma, \mathsf{Q}_y B(a,y) \vdash F}{\Gamma \vdash F}, \tag{DE}$$

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}, \qquad \frac{\Gamma, E(t,z) \vdash E(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} E(x,y)}. \tag{DI-case, DI-progress}$$

The matrix E in DI-progress must be a literal numerical equation. An equation under either binder does not qualify. In DI, z is absent from the retained context, witness, and passive matrix parameters. In DE, a is absent from the retained context, conclusion, and passive matrix parameters. The displayed local assumption is discharged. Bound variables and eigenparameters are renamed before substitution. These rules do not contain a classical rule or induction on a compound formula. Native falsity is the formula $0 = 1$.

Weakening, exchange, contraction, numerical substitution, renaming, and ordinary assumption cut are admissible by induction on derivations. Rename local eigenparameters first. Numerical substitution preserves the root constructor of an equation, so it preserves the progress restriction. For cut, replace each use of the discharged Hyp by its given proof and recurse through the remaining rule instances. These are metatheorems, not uncharged primitive instructions in the checker.

2.3 The equivalent computation-supply presentation

The implementation uses the inherited generated-assumption presentation [2]. It replaces a numerical axiom leaf by

$$\frac{\Gamma, H(w) \vdash A}{\Gamma \vdash A}, \qquad w \notin \mathrm{FV}(\Gamma, A), \quad \Gamma \vdash H(0). \tag{Known-instance supply}$$

The checker generates only the particular equations whose zero instances are justified by J0 or JS and the raw opening lemma; it does not decide arbitrary instances of the displayed side condition. The rule is admissible: substitute zero for w in the premise and cut the proof of $H(0)$. Conversely, a computation theorem E is obtained by supplying the constant equation $H(w) = E$ and immediately using Hyp. Thus the computation-leaf and computation-supply versions prove the same sequents. This proof architecture is inherited, not a new contribution here.

3 Native bootstrap: cases before a full selector

This section checks a dependency that matters for the new scalar basis. Before predecessor is available, the partial selector

$$B(n,a,b) = J(n,a;u.b), \qquad B(0,a,b) = Sa, \quad B(Sn,a,b) = b$$

is not the unrestricted selector $C(0,a,b) = a$, $C(Sn,a,b) = b$. Accordingly, one must not silently invoke the later ordinary-connective macros during the arithmetic bootstrap. In particular, the theorem rule for numerical cases is not by itself a rule for eliminating equations $t = 0$ and $t = Sz$ from a retained context. The finite-case lemma below supplies exactly the stronger form used by the bootstrap.

We use the native Eq/Q/D presentation with hypothesis use, full formula transport, Q elimination, paired elimination, and the two paired introductions. In notation,

$$Q_yF = \text{the clause taking a proof of } F(0) \text{ to } F(t),$$

$$\frac{\Gamma, F(t,0) \vdash F(t,Sz)}{\Gamma \vdash D_{x,y}F(x,y)}, \qquad \frac{\Gamma, E(t,z) \vdash E(t,Sz)}{\Gamma \vdash D_{x,y}E(x,y)}.$$

The second matrix is a literal equation. The usual freshness conditions are retained, and paired elimination opens $Q_yF(a,y)$ at a fresh a. Computation equations may be read as leaves, or obtained from their constant generated-assumption instances and hypothesis use.

3.1 Resources available without a numerical selector

Choose v fresh for t and put $r = J(S0,0;v.t)$. Computation gives $r = t$. Transport this equation along itself with motive $w = t$ to obtain $t = t$. The familiar equality motives then give symmetry, transitivity and numerical congruence; this uses neither predecessor nor successor injectivity.

For any native formula template F, case introduction with an unused witness coordinate gives $D_{x,y}F(y)$ from $\Gamma \vdash F(Sz)$, ignoring the retained $F(0)$. Paired elimination opens $Q_yF(y)$; a supplied proof of $F(0)$ followed by Q elimination at t gives

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)}. \tag{Native NC}$$

The identical construction with progress introduction, using its local $E(z)$, gives contextual literal-equation induction from a base and step. These arguments use no arithmetic selector or ordinary logical connective.

For reference, Q_yF and $D_{x,y}F$, with x unused, are interderivable already: use case introduction and Q elimination in the forward direction, and paired elimination followed by hypothesis use in the reverse direction. Thus the additional Q constructor presents the same internal clauses as the Eq/D formulation.

Even equation explosion is available at this stage. Under $0 = 1$, congruence gives $B(0,0,t) = B(1,0,t)$; computation gives $1 = t$. Consequently every equation follows. External induction on fixed native formula syntax, using case introduction (and the unused-witness representation for Q), gives explosion for every native formula. This is an external proof transformation, not compound-formula induction inside the arithmetic theory.

3.2 Functional finite ranges do not assume branch equations

Say that a particular term b has the *functional range rule* $\mathrm{FR}_R(b)$, for a finite list of numerals R, when the following metarule is derived for every native formula template K:

$$(\Gamma \vdash K(r) \text{ for each } r \in R) \quad \Longrightarrow \quad \Gamma \vdash K(b). \tag{FR}$$

There are no added equations $b = r$ in these premises. The retained context and every fixed parameter of K remain unchanged. In particular, they may mention the original term b.

Use the fixed terms Z, N, j, M defined in Section 4.3, with $Z(t) = B(t,0,0)$. The present proof checks their logical prerequisites; their finite computation identities are verified directly from those definitions.

For $Z(t)$, apply Native NC to the fresh template $w \mapsto K(Z(w))$. The zero branch is $K(1)$ and every successor branch is $K(0)$, by computation and transport. Hence $\mathrm{FR}_{\{0,1\}}(Z(t))$ holds, and therefore so does the same rule for $N(t)$. For $B(t,1,0)$ the identical construction has endpoints two and zero. For

$$j(t) = B(t, 0, B(Z(N(t)), 1, 0))$$

use NC on the outer control: the zero branch is one, and the successor branch has the preceding two-value rule. This proves $\mathrm{FR}_{\{0,1,2\}}(j(t))$, and hence the same rule for $M(t)$, since M is an outer application of j.

The inner parity iterate $U(t) = J(t, 0; u.Z(u))$ has a two-value range rule as well. At counter zero it is one; at a successor it is a value of Z. Apply NC on the counter and the already proved range rule of Z. No induction with a disjunctive invariant is involved.

These constructions justify the cycle equations before contextual finite-case elimination is used. For example, $Z(Z(U(t))) = U(t)$ follows by applying its two-value rule to the equation template $Z(Z(w)) = w$. The equations

$$M(0) = 0, \quad M(1) = 1, \quad M(2) = 2, \qquad M(Sw) = j(M(w))$$

and $j(0) = 1$, $j(1) = 2$, $j(2) = 0$ then follow from the fixed definitions. Their functional range proofs are independent of the ramp invariant.

Lemma 3.1 (Successor cancellation on the three finite values)**.** *Suppose b and c have functional range rules whose values are contained in $\{0,1,2\}$. In any retained context,*

$$Sb = Sc \quad \Longrightarrow \quad b = c$$

is derivable, before global successor injectivity is known.

Proof. Apply the two functional range rules to the equation templates $M(w) = w$ and $j(j(j(w))) = w$. Their required numeral instances are fixed computations. Thus $M(b) = b$, $M(c) = c$, $j^3(b) = b$ and $j^3(c) = c$ are already theorems in the retained context. Apply M to $Sb = Sc$ and use $M(Sw) = j(M(w))$ to get $j(b) = j(c)$. Apply j twice and use the two cycle identities to obtain $b = c$. Every step is ordinary numerical congruence or transport. In particular, no cancellation of an arbitrary successor was inserted. □

Lemma 3.2 (Contextual finite-case elimination for equations)**.** *Let b have $\mathrm{FR}_R(b)$, where R is contained in $\{0,1,2\}$, and let F be the literal equation $u = v$. If, for each $r \in R$, there is a derivation*

$$\Gamma, b = r \vdash u = v,$$

then $\Gamma \vdash u = v$ is derivable. The context may contain arbitrary native formulas, including an induction hypothesis that depends on b.

Proof. Define a native formula template, with both bound variables fresh,

$$K(x) := D_{a,y}(B(y, x, u) = B(y, b, v)),$$

where a is unused. Fix $r \in R$. To introduce $K(r)$ by the case choice with witness zero, its local zero assumption reduces by computation to $Sr = Sb$. Lemma 3.1, with the constant r and the known finite-range term b, yields $b = r$. Cut in the given branch derivation. Its conclusion $u = v$ is exactly the successor matrix, up to the computations of B. Thus $\Gamma \vdash K(r)$.

Apply the functional range rule of b to these closed branch proofs. It gives $\Gamma \vdash K(b)$, whose matrix is $B(y, b, u) = B(y, b, v)$. Paired elimination opens its Q clause. At zero its two sides are Sb, so reflexivity and computation supply the base. Specializing the clause at one gives $u = v$. Close paired elimination. All temporary coordinates are fresh and all temporary assumptions have been discharged. □

The derivation is a finite guard construction, not a primitive implication connective. It relies only on the partial selector, computations, transport, hypotheses, case introduction, paired elimination and Q elimination. Neither predecessor nor the unrestricted selector occurs.

3.3 Audit of the bootstrap and completion of the bridge

The dependency order is now explicit. First establish functional range rules and the two finite cycles using only Native NC and equations. Then establish finite successor cancellation and Lemma 3.2. The finite splits in the ramp induction, the affine scaling induction and the factor-map calculation are instances of that lemma, with $b = N(n)$ or another displayed finite-range term and with a literal-equation conclusion. Retained induction hypotheses are therefore legitimate. The saturation identity $K(c, Su) = SK(c, u)$ is proved directly by NC with no branch assumptions; it does not require an unrestricted contextual split on arbitrary c. No step in this order uses the later ordinary logical macros.

After the corrected predecessor P is constructed, the initializer compiler gives the ordinary scalar I. Consequently

$$C(n, a, b) := I(n, a; u.b)$$

has the unrestricted selector endpoints. The inherited gate, padding, formula-selector and ordinary-connective constructions may now be applied in their stated order. Binder congruence itself uses only case introduction, paired elimination, transport and cut; it is already valid before this final stage.

For clarity, the ordinary interpretations of the native binders are

$$(Q_y B)^\circ = B^\circ(0) \to \forall y\, B^\circ(y), \qquad (D_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)\big).$$

Once the logical macros are available, the native return translations are proved by the same explicit constructions:

$$Q_y B \dashv\vdash B(0) \Rightarrow_* \forall_* y\, B(y),$$

$$D_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big).$$

Forwards, an opened Q clause under its base supplies the arbitrary instance needed by universal introduction. Backwards, a displayed implication and universal supply the successor matrix for case introduction; an unused witness gives the Q case. The existential witness is retained in the D case. External structural induction on finite formulas, together with the endpoint proofs for the ordinary macros, yields both formula composites. Falsity uses the explicit ordinary separation bridge $(0 = 1) \to \bot$.

Finally, the inherited scalar completion and fixed-description PR compiler are used only after the logical interface and initialized I have been recovered. Recurrence uniqueness gives the numerical composites on the whole original domain. The domain is unchanged, interpreted equality is actual equality, and the numerical isomorphism graph is $x = y$. Thus the finite-case repair closes the pre-selector logical dependency; it does not replace the separately supplied complete arithmetic and logical interpretations by standard-model calculations.

4 A successor-seeded scalar basis with two computation schemas

This section completes the explicitly unresolved one-variable successor-seeded candidate in the supplied shared-continuation report [1], in its section "A genuinely smaller numerical grammar: the precise unresolved step." The candidate basis and its conditional initializer compiler are inherited from that report. Unlike the counter-aware successor recursor, its update has just one changing input. The initializer remains a separate argument, but the iterator starts at its successor. The missing result is a numerical predecessor term with a whole-domain proof, obtained here by adapting the supplied scalar 2/3 factor-map construction to shifted arithmetic and correcting a saturated predecessor at three small arguments.

4.1 Syntax, computations, and the precise resources used

The terms are finite staged expressions

$$t ::= x \mid 0 \mid J(n, a; u.h).$$

Only h binds u; its other free variables are passive parameters. Define the abbreviation

$$Sa := J(0, a; u.0), \qquad 1 := S0,$$

and use standard numerals as finite strings of this abbreviation. There are exactly two numerical computation schemas:

$$J(0, a; u.h) = Sa, \tag{J0}$$

$$J(Sn, a; u.h) = h[J(n, a; u.h)/u]. \tag{JS}$$

The first schema, with the abbreviation expanded, says that $J(0, a; u.h) = J(0, a; u.0)$. It does not presuppose a separate successor constructor. No function variables or uniform evaluator are present.

The bootstrap uses equality replacement, numerical cases

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)}, \tag{NC}$$

and contextual induction only on a literal equation E:

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}. \tag{EI}$$

The ordinary eigenvariable conditions apply. In an ordinary intuitionistic first-order presentation, NC follows from coverage $\forall x\,(x = 0 \vee \exists y\, x = Sy)$. In the native paired-binder presentation it is the already available case rule. No use of successor injectivity, predecessor, pairing, an equality characteristic, or induction on a compound formula is made before predecessor is constructed below.

The finite range splits used below require the distinction proved in Section 3. Functional range elimination follows from NC; contextual finite cases with retained induction hypotheses use the finite cancellation and guard construction of Lemma 3.2. The cycle identities are established first by functional range elimination alone. Thus a displayed finite range is not an uncharged disjunction or an assumption of the ordinary-connective macros.

We shall use the following admissible consequence of EI. If two fixed updates are pointwise equal, and the displayed initial values agree, their iterates agree at every counter. Induct on the literal equation between the two iterates: J0 supplies its base and JS supplies its step. This is a derived uniqueness argument, not a primitive extensionality rule for update bodies. All passive arguments are kept fixed while performing the induction.

4.2 A partial selector and shifted addition

Put

$$B(n,a,b) := J(n,a;u.b), \qquad Z(n) := B(n,0,0), \qquad a \oplus b := J(b,a;u.Su).$$

Their computation endpoints are

$$B(0,a,b) = Sa, \quad B(Sn,a,b) = b, \qquad Z(0) = 1, \quad Z(Sn) = 0,$$

$$a \oplus 0 = Sa, \qquad a \oplus Sb = S(a \oplus b).$$

The operation $\oplus$ has standard meaning $a + b + 1$. Ordinary unshifted addition has not been defined and is not used in the proof. Literal-equation induction proves

$$0 \oplus a = Sa, \quad Sa \oplus b = S(a \oplus b), \quad a \oplus b = b \oplus a, \quad (a \oplus b) \oplus c = a \oplus (b \oplus c). \qquad \text{(Shifted algebra)}$$

For the first two equations induct on the second argument. For commutativity induct on b, using the first two equations. For associativity induct on c; at zero both sides are $S(a \oplus b)$, and the successor step applies S to the previous equality. These proofs do not cancel a successor.

In particular, the following are genuine available terms:

$$D(a) := S^2(a \oplus a), \qquad T(a) := S^4((a \oplus a) \oplus a), \qquad R(a) := S^3(a \oplus a). \qquad \text{(Affine terms)}$$

Their standard meanings are respectively $2a + 3$, $3a + 6$, and $2a + 4$. The proof uses their displayed term definitions. From (Shifted algebra) one obtains

$$D(0) = 3, \quad D(Sa) = S^2 D(a), \qquad T(0) = 6, \quad T(Sa) = S^3 T(a). \qquad \text{(Affine shifts)}$$

4.3 Two finite cycles

Define

$$N(n) := Z\big(J(n,0;u.Z(u))\big).$$

The inner iterator is always zero or one: at zero it is one, and at a successor it is a value of Z. The endpoints of Z and finite cases therefore give

$$N(0) = 0, \quad N(Sn) = Z(N(n)), \quad N(S^2 n) = N(n), \qquad N(n) \in \{0,1\}. \qquad \text{(Parity cycle)}$$

This asserts a proved two-state recurrence, not a pre-existing parity operation. For example, if $K(n) = J(n,0;u.Z(u))$, then $N(Sn) = Z(Z(K(n))) = K(n) = Z(N(n))$ by its two cases.

Next put

$$j(u) := B(u,0,B(Z(N(u)),1,0)), \qquad M(n) := j\big(j(J(n,0;u.j(u)))\big).$$

Finite computations give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The range of j is contained in $\{0,1,2\}$: its zero branch is one, and its successor branch is zero or two according to $N(u)$. Consequently the definitions and these three computations give

$$M(0) = 0, \quad M(Sn) = j(M(n)), \quad M(S^3 n) = M(n), \qquad M(n) \in \{0,1,2\}. \qquad \text{(Three-cycle)}$$

Here $M(0) = j(j(1)) = 0$, and the successor equation follows by associativity of composing the fixed term j with itself. The period three follows from its three possible values. No numerical division algorithm has been assumed.

4.4 The shifted ramp and factor map

Set

$$G(n) := J(n, 0; u.u \oplus M(S^2 u)).$$

Its first values are

$$G(0) = 1,\ G(1) = 2,\ G(2) = 4,\ G(3) = 5,\ G(4) = 7,\ G(5) = 8,\ G(6) = 10.$$

Lemma 4.1 (Ramp identities)**.** *The preceding theory proves*

$$M(G(n)) = S(N(n)), \qquad G(S^2 n) = S^3 G(n), \tag{Ramp}$$

and admits the following two finite-case calculations:

	$N(n) = 0$	$N(n) = 1$
$G(Sn)$	$SG(n)$	$S^2 G(n)$
$G(S^3 n)$	$S^4 G(n)$	$S^5 G(n)$.

(Ramp cases)

Proof. Induct on the single equation $M(G(n)) = S(N(n))$. Its base is $M(1) = 1$. If $N(n) = 0$, the induction equation gives $M(G(n)) = 1$, hence $M(S^2 G(n)) = 0$ by the three-cycle. The next state is $G(n) \oplus 0 = SG(n)$, whose M-value is two, as required. If $N(n) = 1$, the current M-value is two and $M(S^2 G(n)) = 1$. The next state is $G(n) \oplus 1 = S^2 G(n)$, whose M-value is one. This proves the invariant and the first line of the case table. Applying that line twice, with the parity cycle, gives the two-step equation; applying it three times gives the last line. □

Lemma 4.2 (Affine identities)**.** *Throughout the entire numerical domain,*

$$N(D(a)) = 1, \qquad N(T(a)) = N(a), \qquad SG(D(a)) = T(a), \tag{Affine parity and ramp}$$

$$G(T(a)) = T(G(a)) \oplus N(a). \tag{Ramp scaling}$$

Proof. For $N(D(a)) = 1$, use EI, $D(0) = 3$, and the two-step parity period. For $N(T(a)) = N(a)$ use EI, $T(0) = 6$, and $T(Sa) = S^3 T(a)$; three parity steps toggle both sides once.

For $SG(D(a)) = T(a)$ use EI on that literal equation. The base is $SG(3) = 6$. The successor step uses $D(Sa) = S^2 D(a)$ and $G(S^2 v) = S^3 G(v)$, so it applies three successors to the previous equation, just as T does.

For (Ramp scaling), the base is $G(6) = 10 = T(1) \oplus 0$. Retain its equation as the induction hypothesis, and split $N(a)$. In the zero case, $N(T(a)) = 0$, so the new left side is $S^4 G(T(a))$. The old right side is $T(G(a)) \oplus 0 = ST(G(a))$, while the new right side is

$$T(SG(a)) \oplus 1 = S^5 T(G(a)).$$

Thus both sides have changed by four successors. In the one case, the new left side is $S^5 G(T(a))$. The old right side is $T(G(a)) \oplus 1 = S^2 T(G(a))$, while the new right side is

$$T(S^2 G(a)) \oplus 0 = S^7 T(G(a)).$$

Thus both sides have changed by five successors. These calculations use only (Shifted algebra), (Affine shifts), and the ramp case table; no successor has been cancelled. □

Define the factor map

$$H(u) := B(Z(N(u)), G(u), u).$$

Its two cases are

$$N(u) = 0 \implies H(u) = u, \qquad N(u) = 1 \implies H(u) = SG(u).$$

Why these shifts are useful. For intuition on standard natural numbers only, $G(u) = \lfloor 3u/2 \rfloor + 1$. Thus H fixes even inputs and sends an odd u to $3(u+1)/2$. After shifting the state by three, an active step multiplies it by $3/2$. The affine maps $D(a) = 2a + 3$ and $T(a) = 3a + 6$ are precisely doubling and tripling in this shifted coordinate: $D(a) + 3 = 2(a + 3)$ and $T(a) + 3 = 3(a + 3)$. Starting at two makes the unshifted base even and fixed by H, while its shifted value is five. The later seed sequence therefore has standard values $5 \cdot 2^x - 3$. This motivates a comparator that tests when its powers of two have been exhausted. The proof below uses the displayed J terms and literal equations; division, exponentiation, and this standard-value description are not prerequisites.

Lemma 4.3 (The factor-map equations)**.** *The following global equations hold:*

$$H(2) = 2, \qquad H(D(a)) = T(a), \qquad H(T(a)) = T(H(a)). \tag{Factor}$$

Proof. The first equation follows from $N(2) = 0$. Since $N(D(a)) = 1$, the second is $SG(D(a)) = T(a)$. For the last equation split $N(a)$, noting that $N(T(a)) = N(a)$. In the zero case both sides equal $T(a)$. In the one case, (Ramp scaling) gives

$$H(T(a)) = SG(T(a)) = S(T(G(a)) \oplus 1) = S^3T(G(a)) = T(SG(a)) = T(H(a)).$$

□

4.5 A comparator without stripping an initializer

Put

$$A(x) := J(x, 0; u.R(u)), \qquad E(x) := SA(x),$$
$$F(x, y) := J(y, A(x); u.H(u)), \qquad L(x, y) := Z(N(F(x, y))).$$

By direct computation and shifted algebra,

$$E(0) = 2, \qquad E(Sx) = D(E(x)), \qquad F(x, 0) = E(x).$$

The definition of F is the decisive initializer alignment: its successor-seeded initial value is exactly $E(x) = SA(x)$. There is no hidden use of predecessor to initialize it at $E(x)$.

Lemma 4.4 (Comparator equations)**.** *The term L is zero or one and satisfies*

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{Comparator}$$

Proof. Its range follows from the range of Z. EI on y proves $F(0, y) = 2$, since $E(0) = 2$ and $H(2) = 2$; hence $L(0, y) = 1$. Also $F(Sx, 0) = E(Sx) = D(E(x))$ has parity one, giving $L(Sx, 0) = 0$.

For the remaining equation first prove, by EI on y with x fixed,

$$F(Sx, Sy) = T(F(x, y)).$$

At zero, the left side is $H(E(Sx)) = H(D(E(x))) = T(E(x))$. In the successor step both iterates apply H, and $H \circ T = T \circ H$ rewrites the result into $T(F(x, Sy))$. Applying $Z \circ N$ and using $N(T(v)) = N(v)$ gives the desired comparator equation.

This proof avoids cancellation in an equation between two successor initializers. Such cancellation would be unavailable at this stage. □

4.6 Saturation and a finite correction give true predecessor

Define conditional increment by

$$K(c,u) := B(c,u,u), \qquad B_m(u) := K(Z(L(Su,m)),u), \qquad V(t,m) := J(t,0;u.B_m(u)).$$

Thus $K(0,u) = Su$ and $K(Sv,u) = u$. The counter m is a passive parameter of the iteration defining V.

Lemma 4.5 (Shifted saturation)**.** *The theory proves*

$$V(St,S^2k) = SV(t,Sk), \qquad V(n,Sn) = Sn, \qquad V(Sn,Sn) = V(S^2n,Sn) = Sn. \quad \text{(Saturation)}$$

Proof. Numerical cases on c give $K(c,Su) = SK(c,u)$. Consequently the comparator equations give

$$B_{Sm}(Su) = SB_m(u), \qquad B_{S^2k}(1) = 2.$$

EI on t, with k fixed, now proves the first displayed equation. At zero its two sides equal two. At a successor, apply the defining update to the induction equation and then use the conjugacy $B_{S^2k}(Su) = SB_{Sk}(u)$.

EI gives $L(Sn,n) = 0$ directly from the comparator equations. A separate EI proves $V(n,Sn) = Sn$: the base is $V(0,1) = 1$, and the successor step uses the previously proved uniform shift at $t = k = n$. This is substitution into a completed lemma, not a change of a passive parameter inside an active induction hypothesis. Finally

$$B_{Sn}(Sn) = K(Z(L(S^2n,Sn)),Sn) = K(Z(L(Sn,n)),Sn) = K(1,Sn) = Sn.$$

One and then two more iterator steps give the last two equations. □

Put

$$Q(m) := J(m,0;u.K(Z(L(S^2u,m)),u)).$$

This term is a capped predecessor rather than the desired predecessor:

$$Q(0) = Q(1) = 1, \qquad Q(S^2k) = Sk. \quad \text{(Capped predecessor)}$$

The first two equations are direct computations. For the last,

$$L(S^2u,S^2k) = L(u,k) = L(Su,Sk)$$

identifies its update with $B_{Sk}(u)$. The derived iterator uniqueness lemma, proved at an independent counter t with k fixed, gives

$$J(t,0;u.K(Z(L(S^2u,S^2k)),u)) = V(t,Sk).$$

Substitute $t = S^2k$ and apply (Saturation).

Now define one fixed correction term

$$\boxed{P(m) := B(Z(L(3,m)),\ Q(Q(m)),\ L(2,m)).} \quad \text{(Predecessor)}$$

There is no arbitrary selector hidden in this expression. Its outer operation is the original partial selector B, whose zero branch adds exactly one successor.

Theorem 4.6 (Predecessor and initializer recovery)**.** *The two-schema successor-seeded scalar theory proves*

$$P(0) = 0, \qquad P(Sn) = n.$$

For every fixed update h, the finite term

$$I(n, a; v.h) := P(J(n, a; u.S(h[Pu/v]))) \qquad \text{(Initializer compiler)}$$

satisfies

$$I(0, a; v.h) = a, \qquad I(Sn, a; v.h) = h[I(n, a; v.h)/v].$$

Proof. The comparator equations give the three direct values

$$P(0) = 0, \qquad P(1) = 0, \qquad P(2) = 1.$$

Indeed, at these inputs $L(3, m) = 0$, so the outer B selects $L(2, m)$, whose three values are zero, zero, one. At $m = S^3k$, one has $L(3, m) = 1$. Thus the zero-control branch of B, together with (Capped predecessor), gives

$$P(S^3k) = SQ(Q(S^3k)) = SQ(S^2k) = S^2k.$$

Numerical cases twice on n now give $P(Sn) = n$ from these displayed equations. Alternatively, two literal-equation inductions, first on $P(S^2n) = Sn$ and then on $P(Sn) = n$, give the same conclusion. No earlier cancellation or injectivity was used. Successor injectivity can now be derived by applying P to an equality of successors.

For the initializer compiler, J0 and the proved predecessor equation give the base $P(Sa) = a$. At a successor, JS exposes the term $S(h[P(J(n, a; u.S(h[Pu/v])))/v])$. The outer P removes its displayed successor and leaves exactly $h[I(n, a; v.h)/v]$. There is no invariant about the internal state to prove: the displayed successor in the update is sufficient. □

4.7 Consequences and the bi-interpretation boundary

The construction supplies the full initialized scalar basis $0, S, I$. In particular the unrestricted selector is now the derived term $C(n, a, b) := I(n, a; u.b)$, with its ordinary endpoints a and b. The supplied scalar arithmetic development can therefore be used from this point onward to obtain actual-equality reflection, induction on arbitrary quantifier-free formulas, pairing and projections, and the compiler for every fixed general primitive-recursion description. Those later arguments are not premises of this bootstrap.

In ordinary iPRA, each fixed J description is interpreted by primitive recursion with seed Sa and its displayed scalar update. Both computation schemas, coverage, separation, and literal-equation induction are then provable. Conversely (Initializer compiler), followed by the existing scalar compiler, interprets every fixed iPRA operation. For each fixed description, literal-equation induction proves uniqueness of its defining recurrence after earlier subdescriptions have been identified. Applied to both composite translations, this identifies each numerical operation with its double translation throughout the entire original domain. Both domain formulas are $x = x$ and both numerical isomorphism graphs are $x = y$.

For the ordinary presentations, full intuitionistic logic is preserved. For the native paired-binder presentation, its ordinary logical macros and the formula return-translation proof must still be included; they apply once the derived selector C is available. This establishes bi-interpretability only

when that logical bridge and the scalar arithmetic completion accompany the present argument. It does not replace them by agreement merely on the standard natural numbers.

The arithmetic advance is completion of the predecessor obligation left open for this exact candidate in the shared-continuation report [1]: the affine factor-map alignment, capped predecessor, and finite correction (Predecessor) supply its missing proof. The candidate grammar and initializer compiler are not new. Finite cycles, a 2/3 factor map, and saturation are adapted from the supplied ordinary scalar predecessor construction. No global literature priority is asserted.

The now-completed basis has two primitive numerical constructors and two computation schemas, with a single update variable. A smaller complete proof checker remains a separate implementation claim, which must charge the actual decoder, scope checks, logical rules, and driver.

5 Recovering full first-order intuitionistic logic

After Theorem 4.6, the derived initialized iterator I and the genuine selector $C(n,a,b) := I(n,a;u.b)$ are available. Consequently $C(0,a,b) = a$ and $C(Sn,a,b) = b$ on the whole domain. The following logical construction is inherited from the supplied native-binder developments[2, 4]; its prerequisites have now been obtained without circularity.

5.1 Gates, padding, and formula selection

First eliminate each $\mathsf{Q}_y B$ by its interderivable representation $\mathsf{D}_{u,y} B$ with unused u. From Q to D use DI-case and QE; from D to Q use DE and the supplied Hyp. Hence it suffices to define the following operations on Eq/D formulas.

For a fresh control v, define a gate by external recursion on formulas:

$$G_v(s = t) := (C(v,0,s) = C(v,0,t)), \qquad G_v(\mathsf{D}_{x,y} A) := \mathsf{D}_{x,y} G_v A.$$

It satisfies $\vdash G_0 A$ and $G_{Sv} A \dashv\vdash A$. Uniform interderivability lifts through D: open the original binder, convert the new base to the old base, use its closure at a successor, convert the result, and introduce the new binder by DI-case. This proves the gate identities by external structural induction. It also proves the padding equivalence

$$A \dashv\vdash \mathsf{D}_{u,y} G_y A \qquad (u, y \notin \mathrm{FV}(A)).$$

The forward direction is DI-case. For the reverse, DE, the zero-gate theorem, and QE at 1 recover A.

Define the formula selector by

$$\mathcal{M}_v(s = t, r = q) := (C(v,s,r) = C(v,t,q)),$$

$$\begin{aligned} \mathcal{M}_v(\mathsf{D}_{x,y} A, \mathsf{D}_{x,y} B) &:= \mathsf{D}_{x,y} \mathcal{M}_v(A,B), \\ \mathcal{M}_v(E, \mathsf{D}_{x,y} B) &:= \mathsf{D}_{x,y} \mathcal{M}_v(G_y E, B), \\ \mathcal{M}_v(\mathsf{D}_{x,y} A, E) &:= \mathsf{D}_{x,y} \mathcal{M}_v(A, G_y E). \end{aligned}$$

Here E is an equation and bound coordinates are freshly aligned. Every recursive call decreases maximum binder depth. Padding and binder congruence prove $\mathcal{M}_0(A,B) \dashv\vdash A$ and $\mathcal{M}_{Sv}(A,B) \dashv\vdash B$, including different input depths.

5.2 Explicit logical macros and their rules

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Its fixed computations give $p(1) = 1, p(2) = 0$. The entire macro list is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}\mathcal{M}_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{u,z}G_z\mathcal{M}_{p(z)}(A, B), \\ \exists_* x A(x) &:= \mathsf{D}_{x,y}G_y A(x), \\ A \vee_* B &:= \exists_* x \mathcal{M}_x(A, B), \\ \forall_* x A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_z A(z), \\ \bot_* &:= (0 = 1). \end{aligned}$$

All auxiliary variables are fresh and u is unused. Implication introduction is DI-case, using the selector's zero endpoint as its local assumption; implication elimination opens the binder and specializes at 1. Existential introduction uses the supplied witness and the gate; existential elimination opens its witness and specializes the closure using the zero-gate theorem.

Conjunction introduction first uses NC to obtain $\mathcal{M}_v(A, B)$ uniformly from the two supplied proofs, then substitutes $v = p(Sz)$ for the successor matrix. Its projections specialize at 1 and 2. Disjunction introduction uses selector witness 0 or 1. For elimination, form the two branch implications into the common conclusion; numerical cases give the implication from an arbitrary selector, and existential elimination supplies that selector.

These derived rules and numerical cases prove $x = 0 \vee_* \exists_* y(x = Sy)$. Universal introduction supplies its zero and successor parts from a proof at a fresh parameter. Universal elimination uses that coverage: its zero branch is the first conjunct, and its successor branch is obtained from the gated closure. Under $0 = 1$, the selector equations give every numerical equation; external structural induction on formulas using DI-case gives every native formula; a Q formula uses its unused-D representation. Thus falsity elimination is available.

For explicitness, the recovered ordinary inference inventory is Hyp, equality reflexivity and replacement, falsity elimination, $\to$I/$\to$E, $\wedge$I and both projections, both $\vee$I rules and $\vee$E, $\forall$I/$\forall$E, and $\exists$I/$\exists$E. The last four are

$$\frac{\Gamma \vdash A(z)}{\Gamma \vdash \forall x A(x)}, \quad \frac{\Gamma \vdash \forall x A(x)}{\Gamma \vdash A(t)}, \quad \frac{\Gamma \vdash A(t)}{\Gamma \vdash \exists x A(x)}, \quad \frac{\Gamma \vdash \exists x A(x) \quad \Gamma, A(z) \vdash F}{\Gamma \vdash F}.$$

Universal introduction excludes z from Γ and passive data; existential elimination also excludes it from F. The propositional rules have their usual reusable-context form: introduce an implication by discharging its antecedent, eliminate it with a proof of that antecedent; form a conjunction from its two proofs and project either; introduce either disjunct and eliminate a disjunction using two local proofs of the same conclusion. No additional classical principle is silently included.

6 The two interpretations and their composites

Let T_J be the ordinary first-order realization: intuitionistic equality logic, J0/JS, literal-equation induction, numerical coverage, and $(0 = 1) \to \bot$. The explicit falsity bridge is essential; the positive equations alone admit a singleton structure. The native theory interprets T_J by the preceding macros, equation induction, coverage, and explosion. Conversely the ordinary meanings of Q, D validate every native rule: DI-case uses coverage, and DI-progress uses induction on its literal equation. The supply presentation is equivalent by substitution and cut.

Appendix A, instantiated with the derived P, C, I, constructs a characteristic of actual equality and translates every quantifier-free formula into a characteristic equation. It therefore recovers all quantifier-free induction. Appendix B constructs pairing and its projections by bounded scalar iteration and compiles each fixed general PR description. Pairing is only a state-coding tool; the interpretation domain remains the whole original domain.

Theorem 6.1 (Full-domain bi-interpretability)**.** *The stated native theory, its computation-supply presentation, and T_J are faithful presentations of a theory bi-interpretable and equiconsistent with full first-order* iPRA*. Both numerical domain formulas are $x = x$; both numerical composite isomorphisms have graph $x = y$.*

Proof. In iPRA, interpret each fixed J body by primitive recursion with base Sa and step h, ignoring the recursion counter in the update. Its two equations and ordinary boundary and induction rules are provable. In the other direction, the initializer compiler of Theorem 4.6, followed by the scalar PR compiler, gives a finite term for each fixed target description. Equality reflection validates the constructor laws and every target induction instance.

For the numerical composites, induct externally on each finite description. Once earlier subdescriptions are identified, the original operation and its double translation have the same base and update. Internal induction on the single equation between their values proves equality at every counter. Parameters remain fixed during this induction. For the original PR descriptions use the corresponding uniqueness proof with their counter argument. This proves internal operation preservation by the identity relation, not merely equality of standard numerical outputs. The same argument gives coherence when substituting parameters changes a staged update description.

For the formula composites, the six displayed ordinary logical macros have their intended meanings by their endpoint and padding proofs. The nontrivial native return translations are

$$\mathsf{Q}_y B \dashv\vdash B(0) \Rightarrow_* \forall_* y B(y),$$

$$\mathsf{D}_{x,y} B \dashv\vdash \exists_* x \big(B(x,0) \Rightarrow_* \forall_* y B(x,y)\big).$$

Forward, DE opens the witness and supplies its closure. Under the local base, QE supplies the body at a fresh parameter; introduce the universal, implication, and existential. Backward, eliminate the existential and use its implication and universal to supply DI-case's successor premise under the retained base. The Q case follows directly or from its unused witness representation. External induction on every finite native and ordinary formula gives both formula composites, in arbitrary contexts.

Finally, ordinary falsity and native $0 = 1$ are contradictions in both directions. The effective finite proof translations therefore give equiconsistency. No theory is asserted to prove its own consistency. □

7 Complete executable checkers and their correspondence

The numerical completion does not by itself shrink a validator. The programs below retain the generated-assumption architecture of the supplied draft and change its arithmetic base equation from $I(0, a) = a$ to $J(0, a) = Sa$. The byte profile also replaces one conditional statement by an equivalent conditional expression. The AST profile uses the exception-based opener and the explicitly decoded iterable aliases described below. Every entire executable module is charged in the comparison, including the JSON input and verdict driver.

7.1 The complete byte profile

```
from json import load
k = 0
i = lambda a, t=None, d=0: [i(b, t, d + b' \x02\x01\x04'[len(a)]) for b in a] if isinstance(a, list)
     else a if k <= a < d else d % 2 and i(t, d=1)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c(m, g)
        a = i(t, s) if m in ([s, u], not s and [t]) and i(t, u) == a else a
        h = m[2:]
    elif t:
        h = a = i(a[2], t, 2)
    else:
        n, t, u = s
        h = [s, i(u, i([*n, t, u], d=1), 1) if n else [t]]
    k -= 1
    c(i(a, [k]), g + [i(h, k if m or len(a) % 2 == m else 0)])
try:
    q = load(open(0)).pop
    c(q(), [])
except:
    print('REJECT')
    raise
print('ACCEPT')
```

The imported JSON parser supplies data only. The submitted data is never evaluated as Python source. No arithmetic oracle, external theorem table, proof producer, helper validator, or unmeasured module is imported. The arithmetic change is the final `[t]` in the assignment of `h`; that is the raw successor of the initializer.

7.2 The complete AST profile

```
from json import load
k = 0

def i(a, t=None, d=0):
    try:
        return [i(b, t, d + b' \x02\x01\x04'[len(a)]) for b in a]
    except:
        return a if k <= a < d else d % 2 and i(t, d=1)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c(m, g)
        if m in ([s, u], not s and [t]) and i(t, u) == a:
            a = i(t, s)
        h = m[2:]
    elif t:
        h = a = i(a[2], t, 2)
    else:
        n, t, u = s
        h = [s, i(u, i([*n, t, u], d=1), 1) if n else [t]]
    k -= 1
    c(i(a, [k]), g + [i(h, k if m or len(a) % 2 == m else 0)])
try:
    q = load(open(0)).pop
    c(q(), [])
except:
    print('REJECT')
    raise
print('ACCEPT')
```

The two checkers have the same canonical mathematical certificates. The latter additionally accepts the safe non-array aliases specified below. The different implementations improve different coordinates; their costs must not be combined into an imaginary single program.

The additional complete file `code/checker_ast_total.py` replaces `for b in a` by `for b in a.copy()` and binds the input pop method with the conventional assignment expression `c((q := load(open(0)).pop)(), [])`. It measures 287/314/711. The input is still loaded once and its final goal is popped before the checker runs. Strings lack `copy` and immediately reject; a nonempty object's string keys likewise reject. Only the empty object adds an alias of []. Opening therefore terminates structurally on finite input trees, including malformed ones. Each non-hypothesis proof invocation consumes a packet. This alternative improves the old AST frontier by one node without the malformed-string tradeoff of the 313-node version. Its source is included in full in the archive.

7.3 Public protocol and exact sorted decoding

The input is one JSON array. Its last item is the closed requested formula. Earlier items are four-field instructions $[m, s, u, t]$, consumed from right to left by `pop`. A major subproof is consumed before its continuation. A hypothesis closes without consuming an instruction. An unread prefix is inert. Successful execution prints `ACCEPT` and exits with status zero; rejection prints `REJECT` and exits

nonzero, possibly with a traceback on standard error. Resource limits can reject sufficiently large valid certificates; mathematical completeness is for unbounded resources.

Arity	Formula	Numerical term	Depth increase
0	truth $0 = 0$	zero	no children
1	conditional closure Q	derived successor	2
2	equality	annotated zero	1
3	paired closure D	successor-seeded J	4

The unary numerical node is an abbreviation for $J(0, a; u.0)$. It is a charged serialization convenience, not an additional primitive computation schema. Numerical binary nodes denote zero but their annotations are still traversed. The first two fields of a ternary formula are likewise checked annotations.

For a protected numerical environment ρ, nearest slot first, the precise nontrivial decoding clauses are

$$\begin{aligned} T_\rho([p]) &= ST_{0,\rho}(p), & F_\rho([B]) &= \mathsf{Q}_y F_{y,\rho}(B), \\ T_\rho([p,q]) &= 0, & F_\rho([s,t]) &= (T_\rho(s) = T_\rho(t)), \\ T_\rho([n,a,h]) &= J(T_{0,0,\rho}(n), T_{0,0,\rho}(a); v.T_{v,v,\rho}(h)), \\ F_\rho([P,U,B]) &= \mathsf{D}_{x,y} F_{y,x,\rho}(B). \end{aligned}$$

Empty arrays decode as zero at numerical sort and truth at formula sort. The two update slots alias the *same* scalar state; they are not separate changing inputs. The two slots of D are distinct numerical binders. The extra slots under a successor and under an iterator's counter and initializer are dummy zeros.

At formula depth $2q$ and numerical depth $2q + 1$, there are q protected slots. Positive coordinates $2j-1$ and $2j$ name the same jth slot; zero denotes numerical zero. Negative coordinates name allocated rigid parameters. The global lower bound k begins at zero and decreases at every non-hypothesis continuation, including computation and neutral endpoint continuations. A major subproof may allocate names before the outer continuation allocates its own name.

The opener $i(a, t, d)$ retains a scalar in $[k, d)$. An exposed numerical coordinate is replaced by the supplied image, itself checked at numerical root, depth one. Exposed formula coordinates denote truth. A missing or invalid *used* numerical image rejects. This numerical-root check is what prevents capture by a surrounding template. The requested goal is checked with no supplied image before any continuation, so meaningful future numerical parameters cannot occur in it.

Canonical data uses arrays and integers. Retained finite fractional coordinates alias their integer floors, and Boolean values have their zero/one meanings. An exceptional scalar can survive only by being replaced with a successfully checked image, or as an exposed formula truth. These conventions do not add numerical operations or axioms.

7.4 The AST profile's additional local aliases

The AST opener traverses its argument directly inside `try`. On a successful used opening, the only additional iterable possibilities are:

$$\texttt{""} \rightsquigarrow [], \qquad \texttt{\{\}} \rightsquigarrow [], \qquad \texttt{\{"":value\}} \rightsquigarrow [[]].$$

The value of the sole dictionary entry is ignored. A nonempty string recurses on a one-character string without reaching a leaf, so cannot finish successfully. A dictionary with a nonempty key therefore cannot complete a used opening either. The empty-key singleton has exactly the length, truth value,

and traversed child of a unary node. All three normalizations commute with scope opening. For the first two aliases the depth increment is immaterial because there are no children.

Actual CPython rejects the nonempty-string case through its finite recursion limit. An ideal interpreter with no recursion bound would diverge on that malformed input. Thus the AST reduction has this explicit raw-input tradeoff; the byte profile retains list dispatch and avoids it. The soundness statement concerns every finite successful run, and the completeness statement promises canonical array/integer certificates.

An empty-key dictionary cannot successfully be a non-hypothesis major: the subsequent slice `m[2:]` fails. As a unary successor counter, its expansion supplies the empty-string zero predecessor, whose numerical opening normalizes to zero. Thus it gives the same computation as the corresponding array. These are local statements about successfully opened and inspected data, not a claim that every ignored field in a raw certificate has a global canonical decoding. Arbitrary unused fields and unread prefixes remain semantically inert.

7.5 Every instruction branch

A truthy major. First recursively prove m; a failed major is never ignored. If $m = [s, u]$ and the target opening of the formula motive t at u matches the requested goal, continue at its opening at s. This is equality transport. If s is falsy and $m = [t]$, the analogous match is QE, with its zero-instance minor. In either case the extra context entry is truth, the empty slice $m[2:]$. An unmatched binary or unary major retains the requested goal and adds only that truth. It does not supply the alleged endpoint equality.

A ternary major cannot match either endpoint candidate. Its slice is $[B]$; opening at the newly allocated name supplies the `Q` assumption of DE, while the requested conclusion is retained. The fresh name is absent from the already checked conclusion and context.

A falsy major and a truthy original fourth field. This is paired introduction with witness t. The opener takes the goal's third field and leaves one induction coordinate protected. Marker $m = []$ chooses its zero-instance assumption. Marker $m = 0$ chooses the current instance only when the opened matrix is a literal equation; otherwise it also chooses the zero instance. The new goal is the successor instance. The test `len(a) % 2 == m` is precisely this distinction on successfully decoded matrices. It never permits progress introduction on a `Q` or `D` matrix.

A zero witness must have a truthy encoding in this branch. In the new basis one can use the term $J(1, 0; u.0)$, which is provably zero. The old initialized-iterator witness $[0, 0, 0]$ now denotes one and cannot be reused as zero. The supplied producer makes the required change. Routine transport along a proved equality handles the selected zero encoding.

A falsy major and falsy original fourth field. The source s must unpack as $[n, a, h]$. The packet's original third field is unused. With a false counter the added raw equation is $[s, [a]]$, expressing the successor-seeded base. With a true counter the inner opening of $[*n, a, h]$ both forces the counter to be unary and validates the predecessor iterator before substitution into the update. This eager check is essential even when that update ignores its state.

Every branch then decrements k, opens the goal with the fresh successor $[k]$, adds the appropriately opened assumption, and uses the same recursive continuation. The major and parity tests select a fresh-name assumption for DE/progress and a zero assumption for cases; computation can generate a local equation containing the fresh name. The admissible known-instance supply rule accounts for precisely that last possibility.

7.6 Why generated J equations are sound

Lemma 7.1 (Structural zero extension)**.** *Read a fixed numerical tree under additional outer slots assigned zero. Alternatively expose those slots to a fresh rigid parameter w and then substitute $w = 0$. Whenever the indicated openings succeed, the decoded numerical terms agree in the theory.*

Proof. Induct externally on the finite tree. Internal binder slots and older rigid parameters are unchanged. An added zero slot, an exposed coordinate, or an alias of w becomes zero. Successor adds a dummy zero on both sides. A checked numerical binary node denotes zero on both sides. At a ternary node its counter and initializer add two zero slots and its update adds two aliases of one symbolic state. Apply the induction hypothesis to all three children. Their fixed J descriptions agree by substitution and congruence. The local iterable aliases in the AST profile normalize before the same argument. This proof does not enumerate the standard natural numbers. □

Lemma 7.2 (The generated equation has a provable zero instance)**.** *If a computation instruction successfully adds $H(w)$, where w is its fresh parameter, then the native theory proves $H(0)$.*

Proof. At a false counter the left side is $J(0, a; v.h)$. Its initializer is read under two dummy zero slots. The initializer on the right side is under one dummy zero, inside its successor, and may expose additional coordinates to w. After $w = 0$, Lemma 7.1 identifies the two initializers. The equation is consequently J0. The particular update does not affect this base value.

At a true counter the eager predecessor check forces a unary $n = [p]$, or its documented empty-key alias. An array counter with two or more fields makes a predecessor tree of arity at least four, which the depth table rejects. A scalar cannot be expanded. The predecessor tree, its initializer, and its update are numerically checked before fresh-name allocation. Thus an out-of-scope coordinate cannot masquerade as its recursive state. The right side substitutes exactly that checked predecessor iterator into the update. The extra dummy zero under the source successor changes no surviving meaningful coordinate, by the same structural argument. Its zero instance is JS. □

The original conclusion and retained context do not contain w. Substitute zero for w in the continuation derivation and cut the proved $H(0)$. The local equation need not be valid for every value of w; its known zero instance is sufficient. This proof is the inherited generated-assumption argument with its base adapted to the new seed.

Theorem 7.3 (Checker correspondence)**.** *Every successful finite execution of each delivered complete checker has a decoded closed conclusion provable in the stated native theory. Conversely, every closed native theorem has a finite canonical certificate accepted by each checker, with unbounded computational resources.*

Proof. For soundness, induct on the finite successful checking trace. Hyp closes a retained formula. A matched transport or QE command is sound by capture-free opening. An unmatched unary or binary major is neutral, and a ternary major performs DE. The DI command supplies exactly one of the two permitted assumptions and maintains its eigenparameter condition. A computation continuation is admissible by Lemma 7.2 and known-instance supply.

Every added context entry is an array produced by opening. For the strict list profile a scalar requested formula cannot close by membership in this list context; its other branches reject or retain it while consuming another instruction. Hence there is no finite successful scalar goal trace. For the AST profile, the additional successfully opened iterables are exactly the local array aliases above. This establishes the required root sorts without an omitted predicate. The initial scope check, the eager numerical-image check, and the decreasing floor maintain the freshness and substitution invariants

throughout the trace. The total-input AST variant has only the empty-object alias, and the assignment expression preserves the input driver's evaluation order.

For completeness, serialize a finite native derivation in the checker's depth-first consumption order. Replace computation leaves with supply of their constant equation followed by Hyp. Use matched endpoints for transport and QE. Use the designated DI markers and a truthy witness encoding, with equality expansion when needed. Canonical descriptions put zero in dummy positions and use allocated rigid names for passive parameters. Allocate a new label at every common continuation, including neutral and arithmetic ones, and account for automatic hypothesis closure. Thus all steps have finite canonical certificates. This is an effective metatheoretic proof translation, not a claim that the small example producer implements the entire translation. □

7.7 Executable evidence and its limits

The main suite contains 36 distinct fixtures: 33 common certificates (17 accepted derivations and 16 rejected inputs), plus three intentional raw-alias boundaries between the profiles. All 216 complete-process runs passed: each fixture was run on all three complete checkers, both normally and with Python `-O`. All 24 targeted mutation controls passed. They detect removal of major-premise checking, admission of compound progress, removal of eager recursive-image checking, and restoration of the incorrect old initialized base equation.

A focused AST suite additionally passed 90 complete-process runs, covering the 33 common cases and 12 explicit alias controls in both execution modes. Some cases intentionally overlap the main suite. The receipts record exact source hashes, expected verdicts, exit codes, and individual results. Accepted examples include active-state JS, induction that uses its actual hypothesis, retained assumptions, nested binders, a nonvacuous paired witness, arbitrary QE specialization, compound-formula transport, and safe neutral endpoint requests.

The total-input AST alternative passes its own 94 complete-process runs: the 33 common cases and 14 focused controls, each normally and with `-O`. Its receipt verifies the exact frozen source and the different alias boundary. The separate numerical diagnostic evaluates the displayed J definitions and checks 306 selected identities and values. It is a finite diagnostic, not the whole-domain proof. The latter is the equation-induction argument in the preceding sections. The named certificate producer demonstrates nonvacuous logical and computation rules, but does not emit the entire predecessor derivation as a kernel certificate. Neither finite testing nor independent assistant review is represented as a machine-checked metatheorem. The archive records this boundary in `CLAIMS.json`.

Run `python3 reproduce.py` for the supplied measurement and proof fixtures, and `python3 build.py` to compile the PDF. The frozen source archives and their hashes allow the corpus measurements and textual novelty search to be repeated without relying on a moving branch.

8 Further research questions

1. Can the shifted comparator and capped-predecessor correction be replaced by a substantially shorter J term? The present proof resolves existence and whole-domain correctness, not optimal term size.
2. Can the native finite-case lifting argument be formalized first, followed by the comparator and predecessor, in Lean or Rocq? This would check the most delicate dependency boundary before formalizing the inherited logical macros and PR compiler.
3. Can the completed numerical basis remove a mechanism from the complete checker, producing a single implementation below all three current size coordinates? The mathematical signature reduction alone does not achieve that stronger benchmark.
4. What lower bounds can be proved after fixing a concrete checker language, input grammar, runtime primitives, and required rejection contract? The observed corpus envelope is not such a theorem.
5. Which other initializer maps $\sigma(a)$, in place of Sa, permit recovery of arbitrary initialization from scalar recursion and equation induction? A useful criterion must supply an internally proved inverse, not assume one in the compiler.
6. Can the capped-predecessor correction be isolated as a general finite-exception repair principle for restricted arithmetic bases? The present term uses only the original partial selector and the comparator.

A From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [5]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Theorem 4.6, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

A.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{1}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{2}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{3}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{4}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p}\,\left[(E(0,\vec{p}) \wedge \forall n(E(n,\vec{p}) \longrightarrow E(Sn,\vec{p}))) \longrightarrow \forall n\, E(n,\vec{p})\right]. \tag{5}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (5) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

A.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{6}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{7}$$

$$K(0) = 1, \qquad K(Sx) = x. \tag{8}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (1). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (7); symmetry and (4) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{9}$$

$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{10}$$

$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{11}$$

To prove the range assertion, split (3). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (2) also give, for all u, v,

$$C(u,v,v) = v. \tag{12}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{13}$$

If $u = 1 = S0$, (8) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

A.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{14}$$

$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{15}$$

Using these equations and (5), we now derive

$$0 + y = y, \tag{16}$$

$$(Sx) + y = S(x + y), \tag{17}$$

$$d(Sx, Sy) = d(x, y), \tag{18}$$

$$d(Sa, a) = 1. \tag{19}$$

Each induction below has a single equation as its induction formula.

For (16), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (17), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (17) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{20}$$

Indeed, both sides are $S(y + r)$.

For (18), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (19). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (18) and the induction hypothesis.

A.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x,y)),\ x,\ y + d(x,y)) = x. \tag{21}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (16) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (12).

For the step, assume (21) at y and abbreviate

$$r = d(x,y), \qquad r' = Pr = d(x,Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (21) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (20) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (21). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa,b) = 1 \quad \longleftrightarrow \quad a = b. \tag{22}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (21). If $d(Sa,b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (19) give $d(Sa,b) = d(Sa,a) = 1$.

Define the equality characteristic by the fixed description

$$e(a,b) = \mathrm{sg}\big(K(d(Sa,b))\big). \tag{23}$$

The sign, countdown, and residual reflection lemmas give

$$e(a,b) = 0 \quad \longleftrightarrow \quad a = b, \tag{24}$$

$$e(a,b) = 0 \ \vee\ e(a,b) = 1. \tag{25}$$

In detail, $e(a,b) = 0$ is equivalent to $K(d(Sa,b)) = 0$ by (10), which is equivalent to $d(Sa,b) = 1$ by (13), and then to $a = b$ by (22). The range assertion is (9) applied to $K(d(Sa,b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a,b) = 0$ branch, (24) gives $a = b$. In the $e(a,b) = 1$ branch, an assumption $a = b$ would give $e(a,b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a\, \forall b\, \big(a = b \vee (a = b \to \bot)\big) \tag{26}$$

is a theorem, not an extra hypothesis.

A.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{27}$$
$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{28}$$
$$b_{A\vee D} = C(b_A, 0, b_D), \tag{29}$$
$$b_{A\to D} = C(b_A, b_D, 0). \tag{30}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{31}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (24)–(25). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (31) for every quantifier-free formula.

A.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (31), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. The same proof first applies to the scalar language, since all the terms used above have the scalar definitions given at the start of this appendix. It yields the QFI needed by the subsequent PR compiler. After that compiler, in the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

B Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [6]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

B.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a,b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

B.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$
$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

B.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section 5 give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

References

[1] VladimirReshetnikov/ProveIt, *A Shared Continuation for iPRA*, `ipra_shared_continuation_2026-11-10.zip`, report section "A genuinely smaller numerical grammar: the precise unresolved step." Pinned corpus commit `2fdcf5bf988bead821c416649f713f98b7ed8776`. Pinned source archive.

[2] VladimirReshetnikov/ProveIt, `ipra_generated_assumptions_2026-11-10.zip`, code, protocol, zero-instance proof, and attributed scalar appendices, at the same commit. Pinned source archive.

[3] VladimirReshetnikov/ProveIt, `ipra_successor_recursion_2026-11-09.zip`, `theory/successor_recursion.tex`, at the same commit. Pinned source archive.

[4] VladimirReshetnikov/ProveIt, `ipra_elimination_assumptions_2026-11-09.zip`, native logical macros and formula return translations, at the same commit.

[5] ProveIt forward-left arithmetic development, as reproduced with attribution in the generated-assumptions archive, `reference/equality_reflection.tex`. Its proof is included here with the instantiated predecessor reference adjusted.

[6] ProveIt progress-closure arithmetic development, as reproduced with attribution in the generated-assumptions archive, `reference/primitive_recursion.tex`. Its scalar compiler is included here with notation local to the appendix.

[7] S. Salehi, *Decidable formulas of intuitionistic primitive recursive arithmetic*, Reports on Mathematical Logic **36** (2002), 55–61. https://rml.tcs.uj.edu.pl/rml-36/36-SALEHI.pdf.

[8] D. E. Severin, *Unary primitive recursive functions*, Journal of Symbolic Logic **73** (2008), 1122–1138. See in particular Remarks 5.10 and 6.9 for related seed shifts. https://arxiv.org/abs/cs/0603063.